

One AI investment-research agent product, six steps from defining the output to acquiring users, all built by me; every step has code or rules you can check.

1 Define the output → 2 Build the agent → 3 Evaluate → 4 Push to users → 5 Guard quality → 6 Acquire and attribute

↳ step 6 feeds back into step 1

1 Define the output: what the agent says and does not say

Without a defined form: walls of narration, the same name repeated, no conclusion up front.

- Output templates: digest, screener, thesis, what-if, earnings
- Three rules: conclusion in the first fold; one page answers one question; a digest is a push, not a dashboard
- Three grounding gates and a frozen feed contract written into the templates, so rules are checkable

4 templates (11,403 lines) adopted into the team library; 3 PRs to a public repo, 2 merged the same day

3 Evaluate: know before release [open source](#)

Prompt changes judged by feel; releases without a regression list.

- Get questions and answers right first: owner writes, peer reviews
- Four tiers with targets; versioned judges calibrated by human audit
- Categorise bad cases (data, tool, ambiguity, reasoning, judge) before feeding back

Set 30 → 100; first pass 5/27, above 0.9 three months later; 145 tests: agent-eval-harness

5 Guard quality: keep bad content and data off the homepage

User-generated content flooded in; nobody can watch the homepage by hand.

- Scoring rule: gates first, universal plus skill dimensions, single attribution, grade bands
- Hourly health check with 10 layers that tell 'empty' from 'broken', baseline and alerts

44 items scored, 19 featured, mean 70.1: scoring rubric, curation SOP, monitor script

2 Build the agent: make the rules run every day

Holdings and watchlists drown users; they only want to know whether anything happened and what it means.

- Rank by event, not by ticker; a move without its own news triggers a peer and sector look-up
- Every line carries a 'what it means, what to watch' lens
- Positions by manual entry, screenshot or broker connection; engine reused for a watchlist sibling in two days

~14K lines of automation, 5 design docs: portfolio-watch, watchlist-watch

4 Push to users: quiet unless something happened [open source](#)

A late push is worthless; a noisy one gets muted.

- No push without an event; re-push only on escalation
- One signal set sliced by ticker, person or book; position-aware ranking by weight × move
- Propose, approve, publish; an audit record per run

Prototype: 16 pushes against 3 quiet runs in a day, near-duplicates within the hour, until a minimum interval fixed it; 61 tests: ticker-digest

6 Acquire and attribute: every ad dollar traceable [open source](#)

Money spent without knowing where users went; algorithms trained on the wrong goal.

- Track before you spend; reconcile GA4 against first-party data
- Split by landing type, not totals
- Every change: diagnose, hypothesis, small budget, compare, kill or scale

Cost per signup down to about a third in three months; use-case pages cheapest and steadiest, roughly 2x the CTR of PMax: ga4-ads-dashboard